GUJARAT TECHNOLOGICAL UNIVERSITY

Shree Swami Atmanand Saraswati Institute of Technology

A

Micro Project Report on

"Student Result Analysis System"

Under subject of

Python For Data Science(BE05000231).

B.E 3rd year, semester - V

COMPUTER ENGINEERING

Under the Guidance of **Prof. Ami D. Patel.**

Submitted by group:

Sr. No.	Name	Enrollment No.
1	**Malankiya Jignesh Pravinbhai**	**240760107109**
2	**Maiyani Om Vijaykumar**	**240760107108**
3	**Kunjadiya Gautam Manishbhai**	**240760107100**
4	**Kunjadiya Meet Shaileshbhai**	**240760107101**
5	**Miyani Neev Ketanbhai**	**240760107125**
6	**Navadiya Ravi**	**240760107130**

Academic Year:2026-2027

SHREE SWAMI ATMANAND SARASWATI INSTITUTE OF TECHNOLOGY

Opp. Kapodra police station, Varachha Road, Surat - 395006

CERTIFICATE

This is to certify that the Project entitled **"Student Result Analysis System"** has been prepared by **Miyani Neev (240760107125), Malankiya Jignesh(240760107109), Maiyani Om(240760107108), Kunjadiya Gautam(240760107100), Kunjadiya Meet(240760107101), Navadiya Ravi (240760107130)** in the subject of Python For Data Science (BE05000231) under my guidance in partial fulfillment of the degree of Bachelor of Engineering in Computer Engineering Department (5th Semester) of Gujarat Technological University, Ahmedabad during the academic year 2026-2027

Date:

Place: SSASIT, Surat

C.R.Patel

Project Guide
Prof. Ami D. Patel

Head of Department
Prof. Chirag R. Patel

Index

Sr. No.	Title	Page No.
1	Cover Page	1
2	Certificate	2
3	Index	3
4	Acknowledgement	4
5	Abstract	5
6	Problem Statement	6
7	Objectives	7
8	Literature Survey	8
9	Existing System & Proposed System	9
10	Technology & System Requirements	10
11	System Design / Architecture	11-12
12	DFD / Functional Modules / Algorithm	12-16
13	Flowchart / Implementation / CRUD / Testing	16-20
14	Results / Conclusion / Future Scope / References	23-28

Acknowledgement

We take this opportunity to express our sincere gratitude to Shree Swami Atmanand Saraswati Institute of Technology, Surat, for providing us with the facilities, academic environment, and support required to successfully complete our micro project on "Student Result Analysis System" under the subject Python for Data Science (BE05000231).

We are deeply thankful to our project guide, Prof. Ami D. Patel, for her valuable guidance, technical suggestions, encouragement, and continuous support throughout the planning, development, testing, and documentation of this project. Her guidance helped us understand important concepts such as data processing, CRUD operations, result calculation, statistical analysis, and data visualization.

We would also like to express our sincere thanks to Prof. Chirag R. Patel, Head of the Computer Engineering Department, for providing a supportive academic environment and encouraging project-based learning.

We are grateful to all the faculty members of the Computer Engineering Department for their valuable support and suggestions. We also thank our friends and family for their continuous motivation and cooperation throughout the project.

Finally, we would like to thank all the project team members for their teamwork, cooperation, and dedicated efforts, which contributed to the successful completion of the Student Result Analysis System.

Abstract

Student marks are usually maintained in spreadsheets, registers, or examination databases. When the number of students increases, manually calculating percentage, grade, pass/fail result, class average, subject-wise averages, and top performance becomes repetitive and can introduce errors. This micro project presents a web-based "Student Result Analysis System" developed as a PBL-2 project for Python for Data Science. The supplied website is titled "Student Result Analysis" and identifies itself as "Python for Data Science • PBL-2 • Full Stack CRUD". The interface provides a Dashboard, CRUD Management, Analysis, and Modules sections.The website manages student information including Student ID, Name, Python marks, Data Science marks, DBMS marks, Mathematics marks, and Computer Networks marks. The front-end calculates percentage, grade, and result status and communicates with backend REST-style endpoints such as /api/students for reading, adding, updating, and deleting records.The system also presents class-level statistics such as total students, class average, pass students, and top score. Subject-wise averages are displayed using horizontal performance bars. Six PBL modules–Data Input, Data Cleaning, Result Calculation, Statistical Analysis, Performance Analysis, and Visualization–are explicitly presented in the website.This report explains the purpose, requirements, design, data flow, modules, algorithms, CRUD integration, testing, outputs, limitations, and future scope of the supplied Student Result Analysis website.

Problem Statement

1.1 Background

Student result processing requires repeated operations on marks. For each student, marks from multiple subjects must be combined, converted into a percentage, mapped to a grade, and checked against the passing condition. At class level, the institution may also need average performance, pass count, top score, and subject-wise comparison.

A simple spreadsheet can perform these calculations, but repeated editing, filtering, and manual formula management can become inconvenient. A web-based system can centralize the operations and provide a consistent interface for adding, viewing, editing, deleting, and analysing student records.

1.2 Problem Statement

The project addresses the problem of developing a simple full-stack web application that can maintain student marks through CRUD operations and automatically analyse the stored marks.

- Student records should be entered through a structured form.
- Marks should be available for five subjects: Python, Data Science, DBMS, Maths, and CN.
- The system should calculate percentage, grade, and result automatically.
- Users should be able to search student records by name or ID.
- Users should be able to update and delete existing student records.
- The dashboard should summarize class performance.
- Subject-wise average performance should be visualized clearly.

1.3 Project Boundary

The supplied source demonstrates the web interface and front-end API calls. The actual server-side implementation/database is not included in

the supplied HTML file, so this report describes the backend contract exposed by the front end rather than claiming a specific database engine.

Objectives

1. To develop a user-friendly web-based Student Result Analysis System.
2. To implement Create, Read, Update, and Delete operations for student records.
3. To maintain marks for Python, Data Science, DBMS, Maths, and CN.
4. To calculate student percentage automatically.
5. To assign grades using predefined percentage ranges.
6. To determine Pass or Fail based on subject-wise minimum marks.
7. To provide search functionality using student name or ID.
8. To display total students, class average, pass students, and top score on a dashboard.
9. To calculate subject-wise average marks.
10. To present six PBL modules covering data input, cleaning, calculation, statistics, performance analysis, and visualization.
11. To provide a responsive interface suitable for desktop and smaller screens.

2.1 Expected Outcome

The expected result is a single web application through which student records can be managed and analysed without manually repeating the same calculations. The interface should give a quick overview of class performance and provide detailed student-level records.

Literature Survey

3.1 Manual Result Processing

Traditional result processing uses registers or spreadsheets. These methods can store marks effectively, but calculation, updating, filtering, and visualization often require additional manual steps.

3.2 Spreadsheet-Based Analysis

Spreadsheets support formulas, sorting, filtering, and charts. However, a web application can provide a controlled interface and predefined calculations so that users do not need to edit formulas directly.

3.3 Web-Based Academic Systems

Web applications can combine a form-based data-entry interface, backend storage, CRUD operations, and analytical dashboards. This architecture is appropriate when records need to be updated and accessed through a common interface.

3.4 Proposed Approach

The supplied website follows this web-application approach. It combines a dashboard, CRUD management form, search field, result table, subjectwise analysis, and six PBL modules in a single page.

Feature	Manual / Register	Spreadsheet	Proposed Website
Add / Edit / Delete	Manual	Possible	Built-in CRUD
Search	Slow	Available	Name / ID search
Percentage	Manual	Formula	Automatic

Grade	Manual	Formula	Automatic
Dashboard	Not available	Can be created	Built in
Visualization	Manual	Possible	Subject bars
Web access	No	Usually filebased	Web interface

Existing System & Proposed System

4.1 Existing Approach

In a basic result-processing approach, student marks are stored and calculations are performed separately. If a record changes, the related calculations and summaries must be checked again.

4.2 Proposed System

The supplied Student Result Analysis website combines record management and analysis. The HTML explicitly provides a CRUD Management section with Add, View, Edit, and Remove operations. The page also provides a Dashboard and Analysis section.

Area	Existing / Basic Method	Proposed Website
Data Entry	Separate register or spreadsheet	Web form
Record Storage	Manual/file based	Backend API contract
Calculation	Repeated formulas	JavaScript function
Grade	Manual/formula	grade(x) function
Result	Manual/formula	Pass/Fail logic
Search	Manual filtering	Live search field
Statistics	Separate calculation	Dashboard
Subject Analysis	Manual chart	Automatic bars
Record Update	Manual	PUT API
Record Delete	Manual	DELETE API

4.3 Main Advantage of the Design

The system places data management and analysis in one interface. A change to a student record can be sent to the backend and then reflected in the table and dashboard after reloading the data.

Technology & System Requirements

5.1 Technologies Evident from the Supplied Website

Technology / Concept	Purpose in the Website
HTML	Page structure and content.
CSS	Layout, colors, cards, buttons, table, responsive design.
JavaScript	CRUD requests, calculation logic, search, rendering and dashboard updates.
REST-style API	Communication with /api/students endpoints.
JSON	Data format used for request/response bodies.
Responsive CSS	Adapts layout at max-width 800px and 550px.

5.2 Hardware Requirements

- Computer or laptop with a modern web browser.
- Internet/network connection when the application is hosted remotely or when backend services are remote.
- Sufficient memory for the browser and backend service.

5.3 Software Requirements

- Modern web browser such as Chrome, Edge, Firefox, or equivalent.
- Web server/backend capable of providing the /api/students endpoints.
- Database or persistent storage at the backend, if configured by the full-stack implementation.
- Development environment for maintaining HTML, CSS, JavaScript and backend code.

System Design

The supplied website uses a single-page dashboard style. Navigation links point to four sections: Dashboard, Manage, Analysis, and Modules. The main content is placed inside a centered container and each section is represented as a card.

6.1 Main Sections

Section	Purpose
Dashboard	Shows total students, class average, pass students and top score.
CRUD Management	Adds, views, searches, edits and deletes student records.
Analysis	Shows subject-wise average marks as performance bars.
Modules	Explains the six PBL modules used by the project.
Footer	Displays project name and PBL-2 / Computer Engineering identification.

6.2 Navigation

The navigation bar uses anchor links to #dashboard, #manage, #analysis and #modules. It is sticky at the top, allowing users to move between major sections without leaving the page.

System Architecture

A conceptual three-layer architecture can be used to explain the supplied full-stack design.

Layer	Responsibility	Examples
Presentation Layer	Displays forms, tables, dashboard, bars and navigation.	HTML + CSS
Application / Logic Layer	Calculates percentage, grade, result and dashboard statistics.	JavaScript functions

Data / API Layer	Provides persistent student records through API endpoints.	/api/students, POST, PUT, DELETE

7.1 Data Flow

Step	Description
1	User enters or selects student information.
2	JavaScript creates a student object.
3	POST or PUT request sends JSON to the backend.
4	Backend returns JSON response.
5	load() fetches /api/students.
6	calc() derives percentage, grade and result.
7	render() updates the student table.
8	dash() updates dashboard statistics and subject bars.

This flow is directly reflected in the supplied JavaScript functions load(), render(), dash(), form(), save(), edit(), del(), and clearForm().

Data Flow Diagram – Level 0

At Level 0, the Student Result Analysis System is represented as one process interacting with the user and a backend data service.

External Entity	Input	System	Output
Student / Faculty / Admin	Student details, marks, search request	Student Result Analysis System	Result table and statistics
User	Add / Update / Delete request	CRUD Processing	Success or error message

Backend API	Student records	Data Processing	JSON student data

8.1 Context Representation

User	→	Student Result Analysis System	→	Backend / Output
Enter marks	→	Validate / calculate	→	Student record
Search	→	Filter / render	→	Matching records
Add / Edit / Delete	→	CRUD API call	→	Updated database state
Analysis	→	Statistics	→	Dashboard / bars

Data Flow Diagram – Level 1

Process	Input	Processing	Output
1.0 Data Input	Form values	Build student object	JSON request
2.0 Data Cleaning	Student data	Backend validates fields	Accepted record
3.0 Result Calculation	Five marks	Percentage, grade, result	Calculated student
4.0 CRUD Management	Add/Edit/Delete	POST/PUT/DELETE	Updated records
5.0 Statistical Analysis	All students	Average, pass count, top score	Dashboard
6.0 Performance Analysis	All marks	Subject averages	Average values
7.0 Visualization	Statistics	Generate bars	Visual output

9.1 Important API Endpoints Visible in the Source

Operation	Method	Endpoint	Purpose

Read	GET	/api/students	Load all students
Create	POST	/api/students	Add a student
Update	PUT	/api/students/{id}	Update selected student
Delete	DELETE	/api/students/{id}	Delete selected student

Functional Modules

10.1 Module 1 – Data Input

The web form accepts Student ID, Name, Python, Data Science, DBMS, Maths, and CN marks. The form() function collects these values and converts marks to numeric values.

10.2 Module 2 – Data Cleaning

The supplied interface describes data cleaning as backend validation of marks and required fields. The HTML itself does not show the backend validation implementation, so the report treats this as the defined module responsibility rather than claiming a specific validation library.

10.3 Module 3 – Result Calculation

The calc(s) function creates an array of five subject marks, calculates their average, assigns a grade, and determines Pass/Fail.

10.4 Module 4 – Statistical Analysis

The dash() function calculates total students, class average, pass count, and top percentage.

10.5 Module 5 – Performance Analysis

The same dashboard function calculates subject-wise averages for Python, Data Science, DBMS, Maths, and CN.

10.6 Module 6 – Visualization

The website represents subject averages using horizontal bars whose widths correspond to the average percentage.

Algorithm

11.1 Student Result Calculation

1. Start.
2. Read the five subject marks: Python, Data Science, DBMS, Maths, and CN.

3. Store the five values in an array.
4. Add the five marks and divide by 5 to calculate percentage.
5. Apply the grade() function to the percentage.
6. Check whether every subject mark is at least 40.
7. If all five subjects are at least 40, set Result = Pass; otherwise set Result = Fail.
8. Return the calculated student object.
9. End.

11.2 Grade Rules in the Supplied Source

Percentage	Grade
90 or above	A+
80-89.99	A
70-79.99	B+
60-69.99	B
50-59.99	C
40-49.99	D
Below 40	F

11.3 Result Rule

The supplied source uses a subject-wise condition: every one of the five marks must be at least 40 for the overall result to be “Pass”. A single subject below 40 therefore makes the result “Fail”.

Flowchart / System Flow

Start	→	Load Students	→	Calculate Result	→	Render Table
	→		→		→	
Render Table	→	Dashboard Statistics	→	Subject Averages	→	Display
	→		→		→	

12.1 CRUD Flow

User Action	JavaScript	API Method	Result
Add Student	save()	POST	New record
Edit Student	edit() then save()	PUT	Updated record
Delete Student	del()	DELETE	Removed record
View / Search	load() + render()	GET	Displayed records

12.2 Search Flow

When text is entered into the Search name/ID field, render() converts the query to lowercase and filters records using the combined Student ID and Name string. The table is then rebuilt using only matching records.

Implementation

13.1 Student Object

```
function form(){   return {
id: $('id').value.trim(),
name: $('name').value.trim(),
python: +$('python').value,
data_science: +$('ds').value,
dbms: +$('dbms').value,
maths: +$('maths').value,      cn:
+$('cn').value
  } }
```

13.2 Result Calculation

```
function grade(x){   return
x>=90?'A+':x>=80?'A':x>=70?'B+':
x>=60?'B':x>=50?'C':x>=40?'D':'F'
}

function calc(s){
let
a=[s.python,s.data_science,s.dbms,s.maths,s.cn];
s.percentage=a.reduce((x,y)=>x+y,0)/5;
  s.grade=grade(s.percentage);
  s.result=a.every(x=>x>=40)?'Pass':'Fail';
return s;
}
```

13.3 Loading Data

```
async function load(){
  data=(await                                    (await
fetch('/api/students')).json()).map(calc);
render();
  dash();
}
```

These snippets are reproduced in shortened form from the supplied HTML source and show the core calculation and data-loading logic.

CRUD & Backend Integration

14.1 Create

When editId is empty, save() sends a POST request to /api/students. The student object is converted into JSON and placed in the request body.

14.2 Read

The load() function sends a GET request to /api/students, converts the returned JSON array using calc(), and then calls render() and dash().

14.3 Update

When a record is edited, edit(id) fills the form and stores the selected ID in editId. The save() function then sends a PUT request to /api/students/{id}.

14.4 Delete

The del(id) function asks for confirmation and then sends a DELETE request to /api/students/{id}.

14.5 Response Handling

```
let r = await fetch(url, {
method: editId ? 'PUT' : 'POST',
  headers: {'Content-Type':'application/json'},
body: JSON.stringify(s)
});
let d = await r.json();
```

The interface displays either d.message or d.error and applies an “ok” or “err” class according to the HTTP response status.

Testing

The following test cases are derived from the functionality explicitly present in the supplied website source. They describe what should be verified when the backend API is available.

ID	Test Case	Expected Result	Status
TC01	Load student list	GET /api/students returns records and table displays them	To be tested with backend

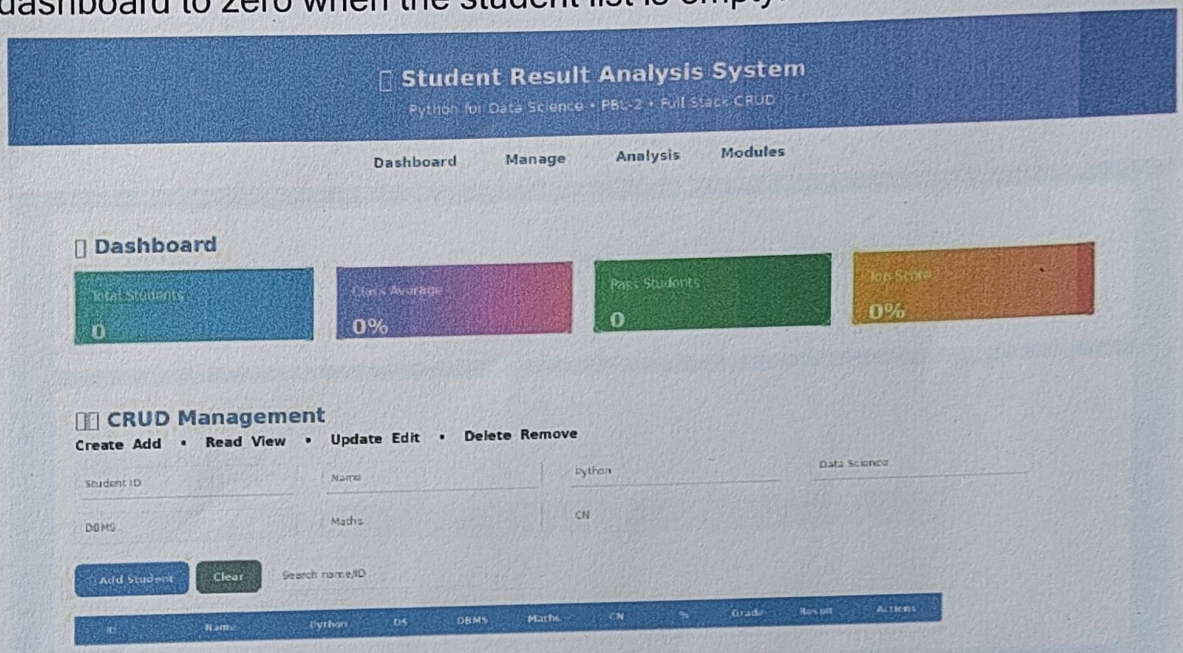
TC02	Add valid student	POST creates record and table reloads	To be tested with backend
TC03	Edit student	PUT updates selected record	To be tested with backend
TC04	Delete student	DELETE removes record after confirmation	To be tested with backend
TC05	Search by name	Only matching names remain visible	Pass in frontend logic
TC06	Search by ID	Only matching ID remains visible	Pass in frontend logic
TC07	Grade calculation	Correct grade according to defined ranges	Pass in source logic
TC08	Fail condition	Any subject below 40 produces Fail	Pass in source logic
TC09	Dashboard	Total, average, pass count and top score update	Pass in source logic
TC10	Subject analysis	Average bar is generated for all five subjects	Pass in source logic

The source file contains the complete front-end logic but does not include the backend implementation, so API-dependent test execution cannot be claimed from the HTML file alone.

Results / Output –

Dashboard Screenshot

The screenshot below is a visual rendering of the supplied website interface in its initial empty-data state. It shows the purple gradient header, sticky navigation, Dashboard cards, and CRUD section exactly reflecting the supplied HTML/CSS structure. The supplied source initializes the dashboard to zero when the student list is empty.



Results / Output –

Figure 15.1: Student Result Analysis – Dashboard and CRUD interface.

CRUD Management Screenshot

The CRUD Management section is the central record-management interface. It provides fields for Student ID, Name, Python, Data Science, DBMS, Maths, and CN, together with Add Student, Clear, search, edit, and delete controls.

Results / Output –

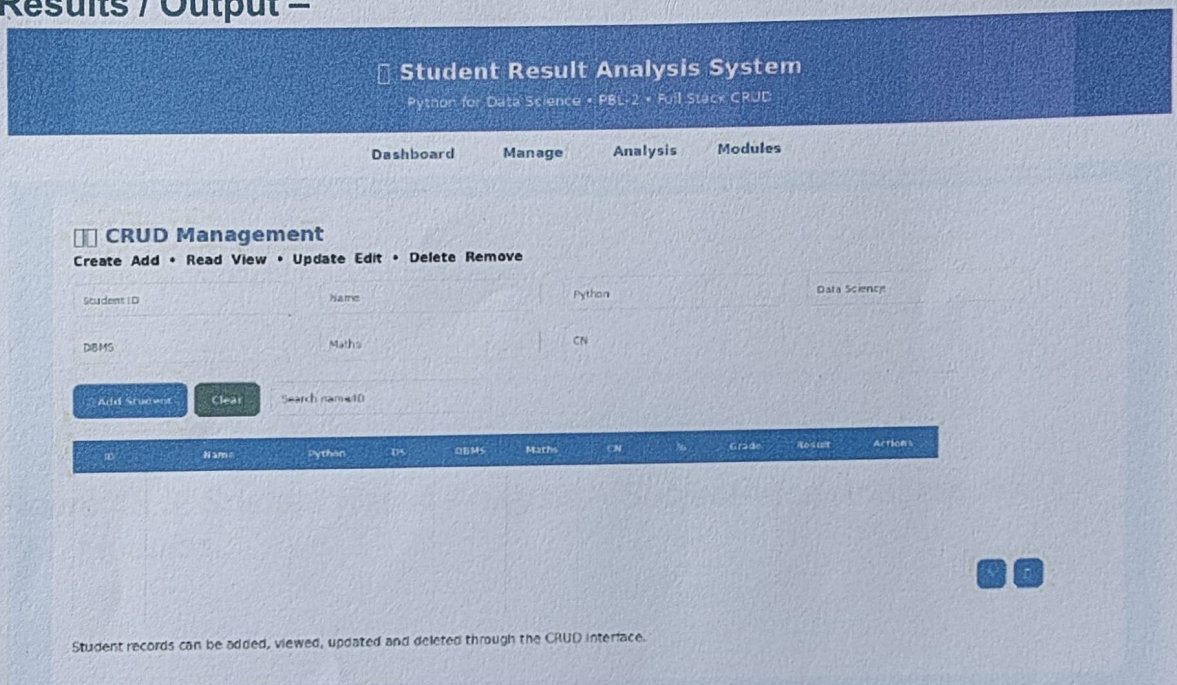


Figure 15.2: Student Result Analysis – CRUD Management interface.

Analysis & Modules Screenshot

The Analysis section displays subject-wise average values using performance bars. The Modules section describes the six PBL modules

Results / Output –

Data Input, Data Cleaning, Result Calculation, Statistical Analysis, Performance Analysis, and Visualization.

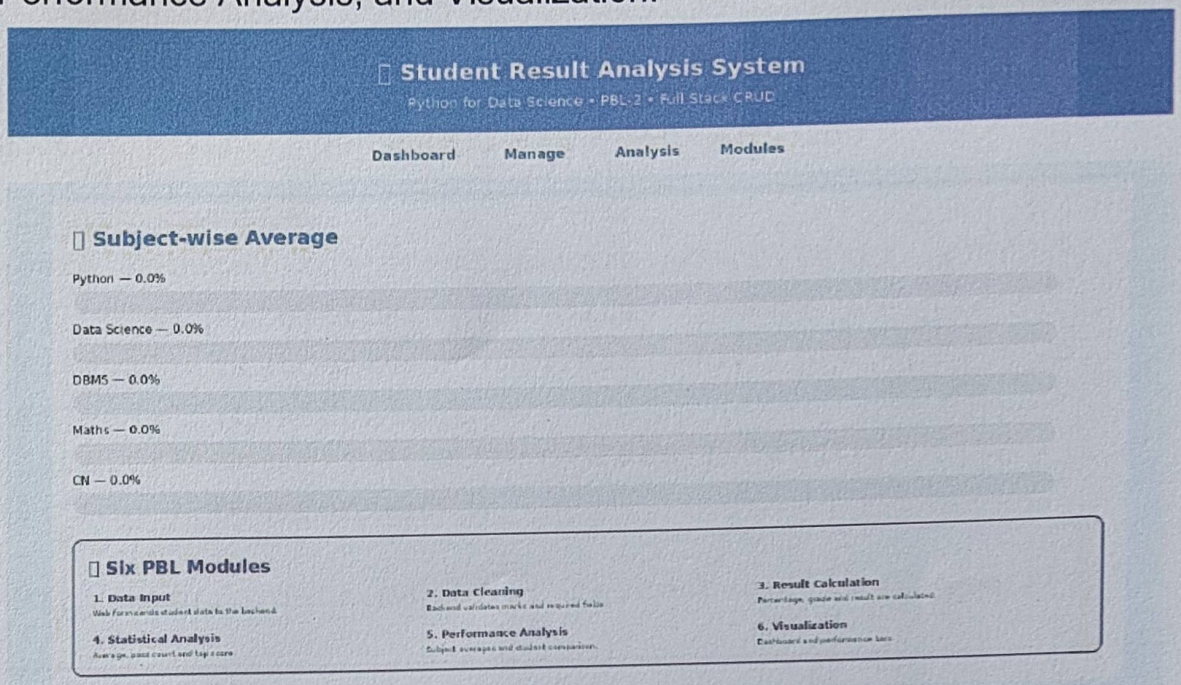


Figure 15.3: Student Result Analysis – Subject-wise Analysis and Six PBL Modules.

Result Calculation & Analysis

16.1 Dashboard Calculations

The dash() function calculates the class average by taking the average of all student percentages. It also counts records whose result is “Pass” and finds the maximum percentage for the Top Score card.

```
let     av  =
data.reduce((a,s)=>a+s.percentage,0)/data.length;
$('total').textContent = data.length;
$('avg').textContent = av.toFixed(1)+'%';
$('pass').textContent =
data.filter(s=>s.result==='Pass').length;
$('top').textContent =

Math.max(...data.map(s=>s.percentage)).toFixed(1)+'%
';
```

16.2 Subject-wise Analysis

The website calculates the average for each of the five subject fields. The result is displayed as a bar with a width corresponding to the calculated average.

16.3 Interpretation

- Total Students represents the number of records loaded from the backend.
- Class Average represents the mean of individual student percentages.
- Pass Students counts students whose result is Pass.
- Top Score is the maximum percentage in the loaded dataset.
- Subject-wise bars allow comparison between average performance in different subjects.

Limitations & Security Considerations

17.1 Limitations

- The supplied HTML file does not include the server-side API or database implementation.
- Actual persistent storage cannot be identified from the HTML source alone.
- Backend validation is described as a module, but its implementation is not included in the supplied file.
- The report cannot claim successful CRUD execution against a live database without the backend service.
- The initial website state may show zero records if /api/students is unavailable or returns an empty list.

17.2 Security Considerations

- Backend APIs should validate Student ID, Name and every marks field before storing data.
- Marks should be restricted to an appropriate numeric range, such as 0-100.
- Authentication and authorization should be added if multiple users can manage records.
- DELETE and UPDATE operations should be protected from unauthorized access.
- Input should be safely handled to prevent injection and malicious payloads.
- API errors should avoid exposing internal server or database details.
- HTTPS should be used when the application is deployed over a network.

17.3 Data Privacy

Student marks and identifiers are academic information. A production deployment should restrict access to authorized users, minimize unnecessary personal information, and protect stored records through appropriate security controls.

Conclusion

The Student Result Analysis System is a practical web-based application for managing and analysing student academic marks. The supplied website combines a dashboard, CRUD management, search, result calculation, statistical analysis, performance visualization, and a sixmodule PBL structure in one interface.

The front-end logic calculates percentage as the average of five subject marks: Python, Data Science, DBMS, Maths, and CN. It assigns grades from A+ to F according to predefined percentage ranges and marks a student as Pass only when all five subject marks are at least 40. The CRUD design provides a clear separation between the user interface and backend API. GET loads records, POST creates a record, PUT updates a record, and DELETE removes a record. After loading, the front end recalculates derived values and refreshes the table and dashboard.

The project demonstrates how data input, validation, calculation, statistical analysis, and visualization can be integrated into a full-stack academic application. With a secure backend and persistent database, it can be extended into a more complete student result-management platform.

Future Scope

- Connect the API to MySQL, PostgreSQL, MongoDB, or another suitable persistent database.
- Add secure login for administrator, faculty, and student roles.
- Add semester, academic year, division, enrollment number, and examination type.
- Support import and export of CSV/Excel result files.
- Generate downloadable PDF result reports.
- Add student-wise progress charts across multiple examinations.
- Add class rank, subject rank, percentile, and performance trend analysis.
- Add attendance and assignment information for broader academic analytics.
- Add validation messages for marks outside the permitted range.
- Add server-side pagination for large student datasets.
- Add database backups and audit logs for record changes.
- Add responsive improvements and accessibility features.
- Add role-based authorization to protect update and delete operations.
- Add automated alerts for students who are below a selected academic threshold.

18.1 Extended Vision

The system can evolve into a complete academic performance portal where authorised faculty members maintain records and students view their own results, while administrators control academic data and reports.

References

1. Python Software Foundation, “Python 3 Documentation.” [Online]. Available: https://docs.python.org/3/
2. Mozilla Developer Network, “JavaScript Guide.” [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide
3. Mozilla Developer Network, “Fetch API.” [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API
4. Mozilla Developer Network, “HTML: HyperText Markup Language.” [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/HTML
5. Mozilla Developer Network, “CSS: Cascading Style Sheets.” [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/CSS
6. R. Fielding et al., “HTTP Semantics,” RFC 9110, IETF, 2022. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9110.html
7. A. Silberschatz, H. F. Korth, and S. Sudarshan, Database System Concepts, 7th ed., McGraw-Hill Education, 2020.